



Universidade do Minho

UNIVERSIDADE DO MINHO
MESTRADO EM CIBERSEGURANÇA
2025/2026

VPN Lab: The Container Version

PG59783 - Carlos Daniel Silva Fernandes - Mestrado em Cibersegurança
PG59790 - Luís Filipe Pinheiro Silva - Mestrado em Cibersegurança
PG59791 - Pedro Augusto Ennes de Martino Camargo - Mestrado em
Cibersegurança

Ao longo do trabalho prático **VPN Lab: The Container Version** foram desenvolvidos e executados vários scripts que implementam as técnicas e os testes propostos no guião. Cada script encontra-se devidamente identificado.

Questão 2.a

Após a execução do script `tun.py`, é criada uma interface virtual denominada `tun`. Através do comando `ip address`, é possível verificar a sua existência e respetivas configurações.

```
root@0a09040545f0:/# ip address
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0@if18: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 86:de:b3:e8:9d:a6 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 10.9.0.5/24 brd 10.9.0.255 scope global eth0
        valid_lft forever preferred_lft forever
3: tun0: <POINTOPOINT,MULTICAST,NOARP> mtu 1500 qdisc noop state DOWN group default qlen 500
    link/none
```

Figura 1: Interface `tun`

Posteriormente, ao alterar o código do script, é possível criar uma interface com um nome personalizado. Neste caso, foi criada a interface `ferna`, correspondente às primeiras cinco letras do apelido *Fernandes*.

```
root@0a09040545f0:/# ip address
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0@if18: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 86:de:b3:e8:9d:a6 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 10.9.0.5/24 brd 10.9.0.255 scope global eth0
        valid_lft forever preferred_lft forever
4: ferna0: <POINTOPOINT,MULTICAST,NOARP> mtu 1500 qdisc noop state DOWN group default qlen 500
    link/none
```

Figura 2: Interface `ferna`

Questão 2.b

Antes de podermos utilizar as interfaces, é necessário configurá-las devidamente. Para tal, deve ser atribuído um endereço IP e a interface deve ser

ativada. Essas operações são realizadas através dos seguintes comandos:

```
ip addr add 192.168.53.99/24 dev tun0
ip link set dev tun0 up
```

Após a execução destes comandos, a interface torna-se totalmente funcional. Podemos verificar a sua configuração utilizando o comando `ip address`, conforme ilustrado na Figura 3.

```
root@0a09040545f0:/# ip address
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0@if18: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 86:de:b3:e8:9d:a6 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 10.9.0.5/24 brd 10.9.0.255 scope global eth0
        valid_lft forever preferred_lft forever
6: tun0: <POINTOPOINT,MULTICAST,NOARP,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UNKNOWN group default qlen 500
    link/none
    inet 192.168.53.99/24 scope global tun0
        valid_lft forever preferred_lft forever
    inet6 fe80::8260:61d5:78e3:4cd/64 scope link stable-privacy
        valid_lft forever preferred_lft forever
```

Figura 3: Interface `tun` após a configuração.

Questão 2.c

Nesta tarefa, procedeu-se à análise do tráfego direcionado para a nossa interface, criando para isso um objeto IP no *Scapy*. Ao executarmos um `ping` para o endereço `192.168.53.1`, é possível observar que a interface está a reencaminhar corretamente os pacotes. Isto acontece porque o endereço IP configurado para a interface `tun` pertence à mesma rede do destino do `ping`, permitindo assim o encaminhamento adequado do tráfego.

```
root@0a09040545f0:/# ping 192.168.53.1
PING 192.168.53.1 (192.168.53.1) 56(84) bytes of data.
```

Figura 4: `Ping` para o endereço `192.168.53.1`.

```

IP / ICMP 192.168.53.99 > 192.168.53.1 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.1 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.1 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.1 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.1 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.1 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.1 echo-request 0 / Raw

```

Figura 5: Encaminhamento correto do tráfego pela interface.

Por outro lado, ao realizar um `ping` para um endereço fora da rede configurada, a interface não consegue responder. Esse comportamento é observado, por exemplo, quando o destino é o endereço `192.168.60.1`, conforme ilustrado nas figuras seguintes.

```

root@0a09040545f0:/# ping 192.168.60.1
PING 192.168.60.1 (192.168.60.1) 56(84) bytes of data.
64 bytes from 192.168.60.1: icmp_seq=1 ttl=64 time=0.146 ms
64 bytes from 192.168.60.1: icmp_seq=2 ttl=64 time=0.104 ms
64 bytes from 192.168.60.1: icmp_seq=3 ttl=64 time=0.109 ms
64 bytes from 192.168.60.1: icmp_seq=4 ttl=64 time=0.110 ms
64 bytes from 192.168.60.1: icmp_seq=5 ttl=64 time=0.117 ms
64 bytes from 192.168.60.1: icmp_seq=6 ttl=64 time=0.092 ms
64 bytes from 192.168.60.1: icmp_seq=7 ttl=64 time=0.102 ms
64 bytes from 192.168.60.1: icmp_seq=8 ttl=64 time=0.118 ms
64 bytes from 192.168.60.1: icmp_seq=9 ttl=64 time=0.102 ms

```

Figura 6: Ping para o endereço `192.168.60.1`.

```

root@0a09040545f0:/volumes# python3 tun.py
Interface Name: tun0
0.0.0.0 > 102.233.89.129 128 frag:6911 / Padding
0.0.0.0 > 102.233.89.129 128 frag:6911 / Padding
0.0.0.0 > 102.233.89.129 128 frag:6911 / Padding

```

Figura 7: A interface não responde a tráfego fora da sua rede.

Questão 2.d

Partindo do código fornecido, foi efetuada uma modificação para que o programa aguardasse especificamente pela recepção de pacotes do tipo ICMP. Após esta alteração, o comportamento observado foi o esperado, como se pode verificar na Figura 8.

```
root@b13bcde1cf11:/# tcpdump -i tun0 -n icmp
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on tun0, link-type RAW (Raw IP), capture size 262144 bytes
21:17:59.261431 IP 192.168.53.99 > 192.168.53.1: ICMP echo request, id 14, seq 1, length 64
21:17:59.264001 IP 1.2.3.4 > 192.168.53.99: ICMP echo reply, id 14, seq 1, length 64
21:18:00.263722 IP 192.168.53.99 > 192.168.53.1: ICMP echo request, id 14, seq 2, length 64
21:18:00.266122 IP 1.2.3.4 > 192.168.53.99: ICMP echo reply, id 14, seq 2, length 64
21:18:01.265850 IP 192.168.53.99 > 192.168.53.1: ICMP echo request, id 14, seq 3, length 64
21:18:01.267863 IP 1.2.3.4 > 192.168.53.99: ICMP echo reply, id 14, seq 3, length 64
21:18:02.267183 IP 192.168.53.99 > 192.168.53.1: ICMP echo request, id 14, seq 4, length 64
21:18:02.269712 IP 1.2.3.4 > 192.168.53.99: ICMP echo reply, id 14, seq 4, length 64
21:18:03.269427 IP 192.168.53.99 > 192.168.53.1: ICMP echo request, id 14, seq 5, length 64
21:18:03.271216 IP 1.2.3.4 > 192.168.53.99: ICMP echo reply, id 14, seq 5, length 64
21:18:04.271951 IP 192.168.53.99 > 192.168.53.1: ICMP echo request, id 14, seq 6, length 64
21:18:04.273717 IP 1.2.3.4 > 192.168.53.99: ICMP echo reply, id 14, seq 6, length 64
21:18:05.274074 IP 192.168.53.99 > 192.168.53.1: ICMP echo request, id 14, seq 7, length 64
21:18:05.276281 IP 1.2.3.4 > 192.168.53.99: ICMP echo reply, id 14, seq 7, length 64
21:18:06.276018 IP 192.168.53.99 > 192.168.53.1: ICMP echo request, id 14, seq 8, length 64
21:18:06.278140 IP 1.2.3.4 > 192.168.53.99: ICMP echo reply, id 14, seq 8, length 64
21:18:07.277835 IP 192.168.53.99 > 192.168.53.1: ICMP echo request, id 14, seq 9, length 64
21:18:07.279917 IP 1.2.3.4 > 192.168.53.99: ICMP echo reply, id 14, seq 9, length 64
21:18:08.279194 IP 192.168.53.99 > 192.168.53.1: ICMP echo request, id 14, seq 10, length 64
21:18:08.281137 IP 1.2.3.4 > 192.168.53.99: ICMP echo reply, id 14, seq 10, length 64
21:18:09.280851 IP 192.168.53.99 > 192.168.53.1: ICMP echo request, id 14, seq 11, length 64
21:18:09.283296 IP 1.2.3.4 > 192.168.53.99: ICMP echo reply, id 14, seq 11, length 64
```

Figura 8: Fluxo na interface tun

Por outro lado, quando não é enviado um pacote ICMP e se escreve manualmente na interface, por exemplo:

```
os.write(tun, b'Hello from the other side!')
```

não ocorre qualquer saída, dado que a interface apenas aceita pacotes IP válidos. A Figura 9 ilustra este comportamento.

```
root@b13bcde1cf11:/# tcpdump -i tun0 -n icmp
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on tun0, link-type RAW (Raw IP), capture size 262144 bytes
21:32:43.517107 IP 192.168.53.99 > 192.168.53.1: ICMP echo request, id 15, seq 4, length 64
21:32:44.541266 IP 192.168.53.99 > 192.168.53.1: ICMP echo request, id 15, seq 5, length 64
21:32:45.565245 IP 192.168.53.99 > 192.168.53.1: ICMP echo request, id 15, seq 6, length 64
21:32:46.589206 IP 192.168.53.99 > 192.168.53.1: ICMP echo request, id 15, seq 7, length 64
21:32:47.613253 IP 192.168.53.99 > 192.168.53.1: ICMP echo request, id 15, seq 8, length 64
21:32:48.637320 IP 192.168.53.99 > 192.168.53.1: ICMP echo request, id 15, seq 9, length 64
```

Figura 9: Ausência de resposta quando não é recebido um pacote IP.

Questão 3

Para conseguirmos enviar o pacote IP através do túnel, foi necessário configurar o **SERVER_IP** para 10.9.0.11 e o **SERVER_PORT** para 9090.

As configurações da nossa interface apenas garantem o encaminhamento na rede 192.168.53.0/24. Para este intervalo de endereços, o servidor VPN recebe o pacote UDP que contém o pacote IP encapsulado no seu interior.

```
root@b13bcde1cf11:/# ping 192.168.53.1
PING 192.168.53.1 (192.168.53.1) 56(84) bytes of data.
```

Figura 10: Ping para 192.168.53.1

```
10.9.0.5:34844 --> 10.9.0.11:9090
  Inside: 192.168.53.99 --> 192.168.53.1
10.9.0.5:34844 --> 10.9.0.11:9090
  Inside: 192.168.53.99 --> 192.168.53.1
10.9.0.5:34844 --> 10.9.0.11:9090
  Inside: 192.168.53.99 --> 192.168.53.1
```

Figura 11: VPN Server

Para realizarmos um ping para a rede 192.168.60.0/24, é necessário encaminhar o tráfego através do router. O comando a utilizar é o seguinte:

```
#ip route add <network> dev <interface> via <router_ip>
os.system("ip route add 192.168.60.0/24 dev {} via 192.168.53.1".format(ifname))
```

```
root@b13bcde1cf11:/# ping 192.168.60.1
PING 192.168.60.1 (192.168.60.1) 56(84) bytes of data.
```

Figura 12: Ping para 192.168.60.1

```

10.9.0.5:47826 --> 10.9.0.11:9090
  Inside: 192.168.53.99 --> 192.168.60.1
10.9.0.5:47826 --> 10.9.0.11:9090
  Inside: 192.168.53.99 --> 192.168.60.1
10.9.0.5:47826 --> 10.9.0.11:9090
  Inside: 192.168.53.99 --> 192.168.60.1

```

Figura 13: VPN Server

Questão 4

Para que o nosso **tun server** consiga efetivamente encaminhar o tráfego, além de ler os pacotes a partir do socket, é necessário que ele crie uma interface na rede 192.168.60.99/24.

Assim, quando pretendemos enviar um ping para o endereço 192.168.60.6, o servidor deve escrever o pacote diretamente nessa interface.

As seguintes imagens ilustram a execução deste processo:

```

root@b13bcde1cf11:/# ping 192.168.60.6
PING 192.168.60.6 (192.168.60.6) 56(84) bytes of data.

```

Figura 14: Ping para 192.168.60.6

```

10.9.0.5:55384 --> 10.9.0.11:9090
  Inside: 192.168.53.99 --> 192.168.60.6
10.9.0.5:55384 --> 10.9.0.11:9090
  Inside: 192.168.53.99 --> 192.168.60.6
10.9.0.5:55384 --> 10.9.0.11:9090
  Inside: 192.168.53.99 --> 192.168.60.6

```

Figura 15: VPN Server

```

root@c1f81e08f3fb:/# tcpdump -ni eth0 icmp
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), capture size 262144 bytes
19:53:02.535415 IP 192.168.53.99 > 192.168.60.6: ICMP echo request, id 7, seq 10, length 64
19:53:02.535446 IP 192.168.60.6 > 192.168.53.99: ICMP echo reply, id 7, seq 10, length 64
19:53:03.558883 IP 192.168.53.99 > 192.168.60.6: ICMP echo request, id 7, seq 11, length 64
19:53:03.558904 IP 192.168.60.6 > 192.168.53.99: ICMP echo reply, id 7, seq 11, length 64
19:53:04.583054 IP 192.168.53.99 > 192.168.60.6: ICMP echo request, id 7, seq 12, length 64
19:53:04.583095 IP 192.168.60.6 > 192.168.53.99: ICMP echo reply, id 7, seq 12, length 64
19:53:05.607391 IP 192.168.53.99 > 192.168.60.6: ICMP echo request, id 7, seq 13, length 64
19:53:05.607430 IP 192.168.60.6 > 192.168.53.99: ICMP echo reply, id 7, seq 13, length 64
19:53:06.631189 IP 192.168.53.99 > 192.168.60.6: ICMP echo request, id 7, seq 14, length 64
19:53:06.631210 IP 192.168.60.6 > 192.168.53.99: ICMP echo reply, id 7, seq 14, length 64
19:53:07.655255 IP 192.168.53.99 > 192.168.60.6: ICMP echo request, id 7, seq 15, length 64
19:53:07.655294 IP 192.168.60.6 > 192.168.53.99: ICMP echo reply, id 7, seq 15, length 64

```

Figura 16: Pacotes a circular na máquina destino (192.168.60.6)

Questão 5

Ao realizar um ping para o endereço 192.168.60.6, não recebemos a resposta *Echo Reply*, uma vez que a nossa VPN não consegue criar um caminho bidirecional. Embora o host 192.168.60.6 envie o *Echo Reply*, este não chega até ao cliente (10.9.0.5).

Para que a comunicação funcione em ambas as direções, o nosso tun server, além de escrever os pacotes na interface `tun`, também precisa de os enviar através de um socket para a outra interface.

É igualmente importante ter em consideração os *file descriptors* que estão à espera de dados. Para esse efeito, utilizamos a função `select` do Linux, que permite monitorizar múltiplos descritores de ficheiros e aguardar até que um deles esteja pronto para leitura.

O seguinte excerto de código, retirado do cliente tun, demonstra a forma como os *file descriptors* são geridos:

```

1 while True:
2     # this will block until at least one interface is ready
3     ready, _, _ = select.select([sock, tun], [], [])
4     for fd in ready:
5
6         if fd is sock:
7             data, (ip, port) = sock.recvfrom(2048)
8             pkt = IP(data)
9             print("From socket <==: {} --> {}".format(pkt.src,
10                 pkt.dst))
11             os.write(tun, bytes(pkt))
12
13         if fd is tun:
14             packet = os.read(tun, 2048)
15             if packet:
16                 pkt = IP(packet)

```



```
16         print("From TUN ==> : {} --> {}".format(pkt.  
17             src, pkt.dst))  
        sock.sendto(packet, (SERVER_IP, SERVER_PORT))
```

Com esta abordagem, garantimos que o tráfego é corretamente encaminhado nas duas direções, tanto do cliente para o servidor, como do servidor para o cliente.

As figuras seguintes comprovam o funcionamento bidirecional:

```
root@b13bcde1cf11:/# ping 192.168.60.6  
PING 192.168.60.6 (192.168.60.6) 56(84) bytes of data.  
64 bytes from 192.168.60.6: icmp_seq=1 ttl=63 time=3.85 ms  
64 bytes from 192.168.60.6: icmp_seq=2 ttl=63 time=2.82 ms  
64 bytes from 192.168.60.6: icmp_seq=3 ttl=63 time=2.80 ms  
64 bytes from 192.168.60.6: icmp_seq=4 ttl=63 time=3.86 ms  
64 bytes from 192.168.60.6: icmp_seq=5 ttl=63 time=2.77 ms  
64 bytes from 192.168.60.6: icmp_seq=6 ttl=63 time=3.22 ms  
64 bytes from 192.168.60.6: icmp_seq=7 ttl=63 time=4.24 ms  
^C  
--- 192.168.60.6 ping statistics ---  
7 packets transmitted, 7 received, 0% packet loss, time 6011ms  
rtt min/avg/max/mdev = 2.765/3.364/4.240/0.564 ms
```

Figura 17: Ping para 192.168.60.6 e *Echo Reply*

```
root@b13bcde1cf11:/volumes# python3 5_tun_client.py
Interface Name: tun0
From TUN ==> : 0.0.0.0 --> 144.114.188.241
From socket <==: 0.0.0.0 --> 135.177.170.82
From TUN ==> : 192.168.53.99 --> 192.168.60.6
From socket <==: 192.168.60.6 --> 192.168.53.99
From TUN ==> : 192.168.53.99 --> 192.168.60.6
From socket <==: 192.168.60.6 --> 192.168.53.99
From TUN ==> : 0.0.0.0 --> 144.114.188.241
From TUN ==> : 192.168.53.99 --> 192.168.60.6
From socket <==: 192.168.60.6 --> 192.168.53.99
From TUN ==> : 192.168.53.99 --> 192.168.60.6
From socket <==: 192.168.60.6 --> 192.168.53.99
From TUN ==> : 192.168.53.99 --> 192.168.60.6
From socket <==: 192.168.60.6 --> 192.168.53.99
From TUN ==> : 192.168.53.99 --> 192.168.60.6
From socket <==: 192.168.60.6 --> 192.168.53.99
From TUN ==> : 192.168.53.99 --> 192.168.60.6
From socket <==: 192.168.60.6 --> 192.168.53.99
```

Figura 18: Cliente tun a processar os pacotes

```

root@1ae3a198de70:/volumes# python3 5_tun_server.py
Interface Name: tun0
From TUN ==> : 0.0.0.0 --> 135.177.170.82
From socket <==: 0.0.0.0 --> 144.114.188.241
From TUN ==> : 0.0.0.0 --> 135.177.170.82
From socket <==: 192.168.53.99 --> 192.168.60.6
From TUN ==> : 192.168.60.6 --> 192.168.53.99
From socket <==: 192.168.53.99 --> 192.168.60.6
From TUN ==> : 192.168.60.6 --> 192.168.53.99
From socket <==: 0.0.0.0 --> 144.114.188.241
From socket <==: 192.168.53.99 --> 192.168.60.6
From TUN ==> : 192.168.60.6 --> 192.168.53.99
From socket <==: 192.168.53.99 --> 192.168.60.6
From TUN ==> : 192.168.60.6 --> 192.168.53.99
From socket <==: 192.168.53.99 --> 192.168.60.6
From TUN ==> : 192.168.60.6 --> 192.168.53.99
From socket <==: 192.168.53.99 --> 192.168.60.6
From TUN ==> : 192.168.60.6 --> 192.168.53.99
From socket <==: 192.168.53.99 --> 192.168.60.6
From TUN ==> : 192.168.60.6 --> 192.168.53.99

```

Figura 19: Servidor tun a enviar e receber pacotes

6	3.200944544	192.168.60.6	192.156.2.23	ICMP	98 Echo (ping) request	id=0x0026, seq=2/512, ttl=63
7	4.224640970	192.168.60.6	192.156.2.23	ICMP	98 Echo (ping) request	id=0x0026, seq=3/768, ttl=63
8	5.248427651	192.168.60.6	192.156.2.23	ICMP	98 Echo (ping) request	id=0x0026, seq=4/1024, ttl=63
9	6.271955585	192.168.60.6	192.156.2.23	ICMP	98 Echo (ping) request	id=0x0026, seq=5/1280, ttl=63
10	16.609059779	10.9.0.5	10.9.0.11	UDP	90 34737 → 9090	Len=48
11	17.792621862	10.9.0.11	10.9.0.5	UDP	90 9090 → 34737	Len=48

Figura 20: Captura no Wireshark evidenciando o tráfego bidirecional

```

root@c1f81e08f3fb:/# tcpdump -ni eth0 icmp
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), capture size 262144 bytes
22:15:40.236253 IP 192.168.53.99 > 192.168.60.6: ICMP echo request, id 40, seq 1, length 64
22:15:40.236290 IP 192.168.60.6 > 192.168.53.99: ICMP echo reply, id 40, seq 1, length 64
22:15:41.237695 IP 192.168.53.99 > 192.168.60.6: ICMP echo request, id 40, seq 2, length 64
22:15:41.237733 IP 192.168.60.6 > 192.168.53.99: ICMP echo reply, id 40, seq 2, length 64
22:15:42.239347 IP 192.168.53.99 > 192.168.60.6: ICMP echo request, id 40, seq 3, length 64
22:15:42.239382 IP 192.168.60.6 > 192.168.53.99: ICMP echo reply, id 40, seq 3, length 64
22:15:43.240745 IP 192.168.53.99 > 192.168.60.6: ICMP echo request, id 40, seq 4, length 64
22:15:43.240782 IP 192.168.60.6 > 192.168.53.99: ICMP echo reply, id 40, seq 4, length 64
22:15:44.241789 IP 192.168.53.99 > 192.168.60.6: ICMP echo request, id 40, seq 5, length 64
22:15:44.241819 IP 192.168.60.6 > 192.168.53.99: ICMP echo reply, id 40, seq 5, length 64
22:15:45.243958 IP 192.168.53.99 > 192.168.60.6: ICMP echo request, id 40, seq 6, length 64
22:15:45.243992 IP 192.168.60.6 > 192.168.53.99: ICMP echo reply, id 40, seq 6, length 64
22:15:46.245316 IP 192.168.53.99 > 192.168.60.6: ICMP echo request, id 40, seq 7, length 64
22:15:46.245349 IP 192.168.60.6 > 192.168.53.99: ICMP echo reply, id 40, seq 7, length 64

```

Figura 21: Pacotes a circular na máquina de destino (192.168.60.6)

Questão 6

Durante a experiência, estabelecemos uma ligação **telnet** entre o Host U e o Host V (**telnet 192.268.60.5 23**). Enquanto a ligação estava ativa, parámos o programa **5_tun_server.py** para interromper o túnel VPN.

Quando o túnel foi interrompido, a ligação Telnet deixou de funcionar e já não conseguíamos escrever na sessão. Depois de voltarmos a iniciar os programas **5_tun_client.py** e **5_tun_server.py**, e restabelecer o túnel, a ligação Telnet voltou a funcionar normalmente, permitindo-nos continuar a escrever e executar comandos.

Questão 7

Na máquina 192.168.60.5, se tentarmos executar um **ping** para 10.9.0.5, a comunicação não será possível se as tabelas de routing não estiverem configuradas corretamente. Ao apagar a entrada **default**, deixa de ser possível executar o **ping**. No entanto, ao introduzir o seguinte comando:

```
ip route add 10.9.0.5 via 192.168.60.11 dev eth0
```

é possível restabelecer a comunicação entre as máquinas 192.168.60.5 e 10.9.0.5. Este comando define que o tráfego destinado a 10.9.0.5 deve ser encaminhado através do gateway 192.168.60.11.

```

root@c1f81e08f3fb:/# ip route del default
root@c1f81e08f3fb:/# ping 10.9.0.5
ping: connect: Network is unreachable
root@c1f81e08f3fb:/# ip route add 10.9.0.5 via 192.168.60.11 dev eth0
root@c1f81e08f3fb:/# ping 10.9.0.5
PING 10.9.0.5 (10.9.0.5) 56(84) bytes of data.
64 bytes from 10.9.0.5: icmp_seq=1 ttl=63 time=2.11 ms
64 bytes from 10.9.0.5: icmp_seq=2 ttl=63 time=2.29 ms
64 bytes from 10.9.0.5: icmp_seq=3 ttl=63 time=2.17 ms
64 bytes from 10.9.0.5: icmp_seq=4 ttl=63 time=2.95 ms
64 bytes from 10.9.0.5: icmp_seq=5 ttl=63 time=2.41 ms

```

Figura 22: Routing

Questão 8

De forma a criar um túnel VPN entre duas redes privadas, é necessário ter dois servidores VPN, cada um deles ligado a uma das redes, que possam comunicar entre si. Para tal usamos o programa `5_tun_server.py` desenvolvido na questão 5, alterando apenas o comando ***ip route add <network> dev <interface> via <router ip>*** de cada *VPN server* para:

- *ip route add 192.168.60.0 dev <interface> via 192.168.53.2*, no servidor sem acesso a rede 192.168.60.0, ou seja, no servidor com ip 10.9.0.12
- *ip route add 192.168.50.0 dev <interface> via 192.168.53.1*, no servidor sem acesso a rede 192.168.50.0, ou seja, no servidor com ip 10.9.0.11

De forma a automatizar o processo de inicializar os servidores, criamos dois programas: `8_tun_client` e `8_tun_server`, sendo estes executados no servidor ligado a rede 192.168.50.0 e no servidor ligado a rede 192.168.60.0 respectivamente.

```

root@796c334950b7:/# ping 192.168.60.6
PING 192.168.60.6 (192.168.60.6) 56(84) bytes of data.
64 bytes from 192.168.60.6: icmp_seq=1 ttl=62 time=4.27 ms
64 bytes from 192.168.60.6: icmp_seq=2 ttl=62 time=2.15 ms
64 bytes from 192.168.60.6: icmp_seq=3 ttl=62 time=2.49 ms
^C
--- 192.168.60.6 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2035ms
rtt min/avg/max/mdev = 2.150/2.971/4.274/0.931 ms
root@796c334950b7:/#

```

Figura 23: Ping de 192.168.50.6 para 192.168.60.6 e *Echo Reply*

```

root@a0b0be3ed661:/volumes# 6_tun_server.py
Interface Name: tun0
From socket <==: 192.168.50.6 --> 192.168.60.6
From TUN ==> : 192.168.60.6 --> 192.168.50.6
From socket <==: 192.168.50.6 --> 192.168.60.6
From TUN ==> : 192.168.60.6 --> 192.168.50.6
From socket <==: 192.168.50.6 --> 192.168.60.6
From TUN ==> : 192.168.60.6 --> 192.168.50.6

```

Figura 24: Servidor da rede 192.168.60.0 a enviar e receber pacotes

```

root@740f811b9806:/volumes# 6_tun_client.py
Interface Name: tun0
From TUN ==> : 192.168.50.6 --> 192.168.60.6
From socket <==: 192.168.60.6 --> 192.168.50.6
From TUN ==> : 192.168.50.6 --> 192.168.60.6
From socket <==: 192.168.60.6 --> 192.168.50.6
From TUN ==> : 192.168.50.6 --> 192.168.60.6
From socket <==: 192.168.60.6 --> 192.168.50.6

```

Figura 25: Servidor da rede 192.168.50.0 a enviar e receber pacotes

1	2025-11-10 09:3	10.9.0.12	10.9.0.11	UDP	126 9090 ~ 9090 Len=84
2	2025-11-10 09:3	10.9.0.11	10.9.0.12	UDP	126 9090 ~ 9090 Len=84
3	2025-11-10 09:3	10.9.0.12	10.9.0.11	UDP	126 9090 ~ 9090 Len=84
4	2025-11-10 09:3	10.9.0.11	10.9.0.12	UDP	126 9090 ~ 9090 Len=84
5	2025-11-10 09:3	10.9.0.12	10.9.0.11	UDP	126 9090 ~ 9090 Len=84
6	2025-11-10 09:3	10.9.0.11	10.9.0.12	UDP	126 9090 ~ 9090 Len=84

Figura 26: Captura no Wireshark a demonstrar o túnel VPN entre as duas redes privadas

Questão 9

A criação de uma interface **tap** faz-se da mesma forma que a criação de uma interface **tun**. Devido a isto, podemos reutilizar o script de criação usado na questão 2, sendo apenas necessário alterar o argumento *IFF_TUN*, usado na função *struct.pack*, para *IFF_TAP* e alterar a lógica de captura de **packets** para utilizar objetos **Ether**.

Para verificar se a interface está a funcionar, fizemos ping para o IP 192.168.53.1, verificando que o programa captura o pedido ARP criado pelo comando ping.

```
root@a0b0be3ed661:/# ping 192.168.53.1
PING 192.168.53.1 (192.168.53.1) 56(84) bytes of data.
From 192.168.53.99 icmp_seq=1 Destination Host Unreachable
From 192.168.53.99 icmp_seq=2 Destination Host Unreachable
From 192.168.53.99 icmp_seq=3 Destination Host Unreachable
^C
--- 192.168.53.1 ping statistics ---
5 packets transmitted, 0 received, 100% packet loss, time 4092ms
pipe 4
```

Figura 27: Ping de um host na rede 192.168.53.0

```
root@a0b0be3ed661:/volumes# python3 tap.py
Interface Name: tap0
Ether / ARP who has 192.168.53.1 says 192.168.53.99
Ether / ARP who has 192.168.53.1 says 192.168.53.99
Ether / ARP who has 192.168.53.1 says 192.168.53.99
Ether / ARP who has 192.168.53.1 says 192.168.53.99
Ether / ARP who has 192.168.53.1 says 192.168.53.99
Ether / ARP who has 192.168.53.1 says 192.168.53.99
```

Figura 28: Captura do pedido ARP na interface tap

Posteriormente modificamos o loop de captura de packets de forma a que, quando este recebesse um **ARP request**, o programa criasse uma spoofed **ARP reply** e a escrevesse na interface **tap**.

Para testar se o programa estava a funcionar, usamos o comando **arping** para descobrir a **MAC address** dos IPs 192.168.53.33 e 1.2.3.4 através de **ARP requests**. As figuras seguintes demonstram os resultados desses testes:

```
root@a0b0be3ed661:/# arping -I tap0 192.168.53.33
ARPING 192.168.53.33
42 bytes from aa:bb:cc:dd:ee:ff (192.168.53.33): index=0 time=2.146 msec
42 bytes from aa:bb:cc:dd:ee:ff (192.168.53.33): index=1 time=2.246 msec
42 bytes from aa:bb:cc:dd:ee:ff (192.168.53.33): index=2 time=3.962 msec
42 bytes from aa:bb:cc:dd:ee:ff (192.168.53.33): index=3 time=2.494 msec
^C
--- 192.168.53.33 statistics ---
4 packets transmitted, 4 packets received, 0% unanswered (0 extra)
rtt min/avg/max/std-dev = 2.146/2.712/3.962/0.733 ms
```

Figura 29: Uso do comando arping para descobrir a MAC address do ip 192.168.53.33

```

root@a0b0be3ed661:/volumes# python3 tap.py
Interface Name: tap0
-----
Ether / ARP who has 192.168.53.33 says 192.168.53.99 / Padding
***** Fake response: Ether / ARP is at aa:bb:cc:dd:ee:ff says 192.168.53.33
-----
Ether / ARP who has 192.168.53.33 says 192.168.53.99 / Padding
***** Fake response: Ether / ARP is at aa:bb:cc:dd:ee:ff says 192.168.53.33
-----
Ether / ARP who has 192.168.53.33 says 192.168.53.99 / Padding
***** Fake response: Ether / ARP is at aa:bb:cc:dd:ee:ff says 192.168.53.33
-----
Ether / ARP who has 192.168.53.33 says 192.168.53.99 / Padding
***** Fake response: Ether / ARP is at aa:bb:cc:dd:ee:ff says 192.168.53.33

```

Figura 30: Captura do pedido ARP pelo programa e spoofing da respetiva ARP reply

```

root@a0b0be3ed661:/# arping -I tap0 1.2.3.4
ARPING 1.2.3.4
42 bytes from aa:bb:cc:dd:ee:ff (1.2.3.4): index=0 time=2.574 msec
42 bytes from aa:bb:cc:dd:ee:ff (1.2.3.4): index=1 time=2.988 msec
42 bytes from aa:bb:cc:dd:ee:ff (1.2.3.4): index=2 time=2.435 msec
42 bytes from aa:bb:cc:dd:ee:ff (1.2.3.4): index=3 time=3.043 msec
^C
--- 1.2.3.4 statistics ---
4 packets transmitted, 4 packets received, 0% unanswered (0 extra)
rtt min/avg/max/std-dev = 2.435/2.760/3.043/0.261 ms

```

Figura 31: Uso do comando arping para descobrir a MAC address do ip 1.2.3.4

```

root@a0b0be3ed661:/volumes# python3 tap.py
Interface Name: tap0
-----
Ether / ARP who has 1.2.3.4 says 192.168.53.99 / Padding
***** Fake response: Ether / ARP is at aa:bb:cc:dd:ee:ff says 1.2.3.4
-----
Ether / ARP who has 1.2.3.4 says 192.168.53.99 / Padding
***** Fake response: Ether / ARP is at aa:bb:cc:dd:ee:ff says 1.2.3.4
-----
Ether / ARP who has 1.2.3.4 says 192.168.53.99 / Padding
***** Fake response: Ether / ARP is at aa:bb:cc:dd:ee:ff says 1.2.3.4
-----
Ether / ARP who has 1.2.3.4 says 192.168.53.99 / Padding
***** Fake response: Ether / ARP is at aa:bb:cc:dd:ee:ff says 1.2.3.4

```

Figura 32: Captura do pedido ARP pelo programa e spoofing da respetiva ARP reply